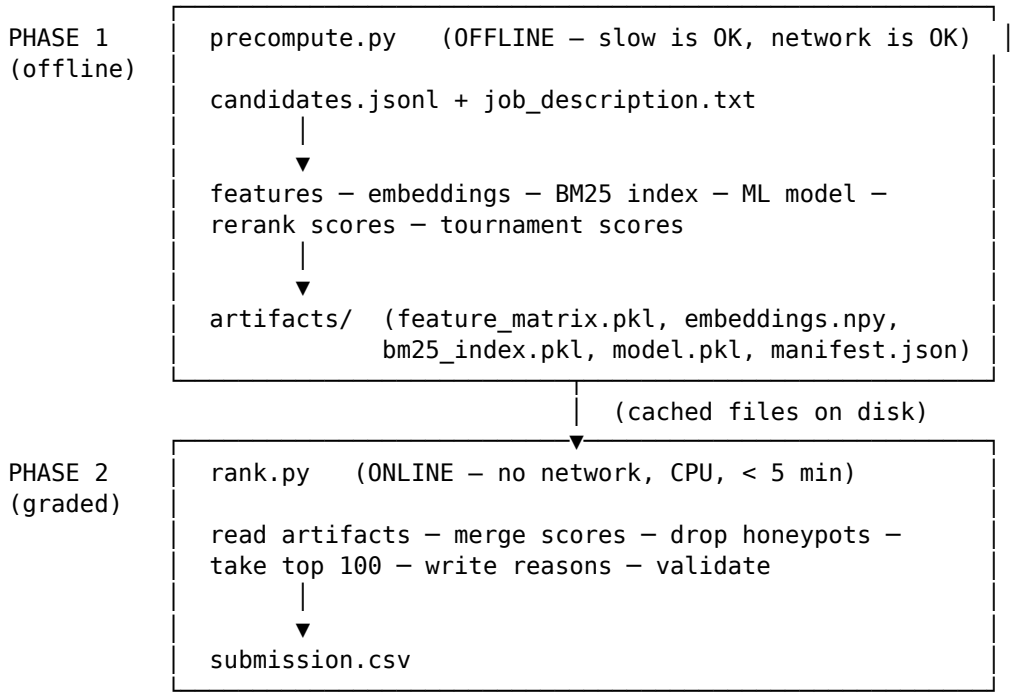
02 — Architecture

The one idea that makes everything work: two phases

The graded ranking step has to run with **no network, on CPU, in under 5 minutes**. But the *smart* parts of ranking — embedding 100,000 profiles with a neural model, training a gradient-boosted ranker, running a cross-encoder — are slow and sometimes need the network. You can't do both in one step.

So the system is split in two:



Everything expensive happens in **Phase 1** and is written to the `artifacts/` folder. **Phase 2** only *reads* those files and does cheap arithmetic, so it comfortably meets the constraints. This is why adding more "intelligence" (a reranker, a tournament, GitHub evidence) never slows down the graded step — it all lands in Phase 1.

Phase 1 in detail (`precompute.py`)

```
1. Hash inputs ──► if --resume and nothing changed, skip everything.
2. Deconstruct the JD ──► a JobSpec (required skills, seniority, location).
                          (the only place an LLM may run; falls back to rules)
3. Stream candidates once ──► build:
      • the 40-feature row per candidate   (pipeline/feature_engineering.py)
      • the full text doc (for search)      (pipeline/loader.corpus_text)
      • a focused text doc (for rerank)     (pipeline/loader.rerank_text)
4. Apply hard gates ──► mark honeypots / unqualified (still kept in the matrix,
                        flagged, so the API can show *why* they were excluded).
5. Embeddings ──► encode all docs (sentence-transformers or hashing). Cached.
6. BM25 index ──► build keyword index. Score the JD query against it.
7. Retrieval ──► fuse BM25 + embedding cosine with Reciprocal Rank Fusion.
8. Accuracy layer:
      • graded pseudo-labels ──► train the LambdaMART ranker (saved to model.pkl)
      • cross-encoder rerank of the top ~300 ──► rerank_score column
      • Bradley-Terry tournament of the top ~60 ──► tournament_score column
9. Persist ──► feature_matrix.pkl, embeddings.npy, bm25_index.pkl, model.pkl,
               manifest.json, candidate_ids.txt.
```

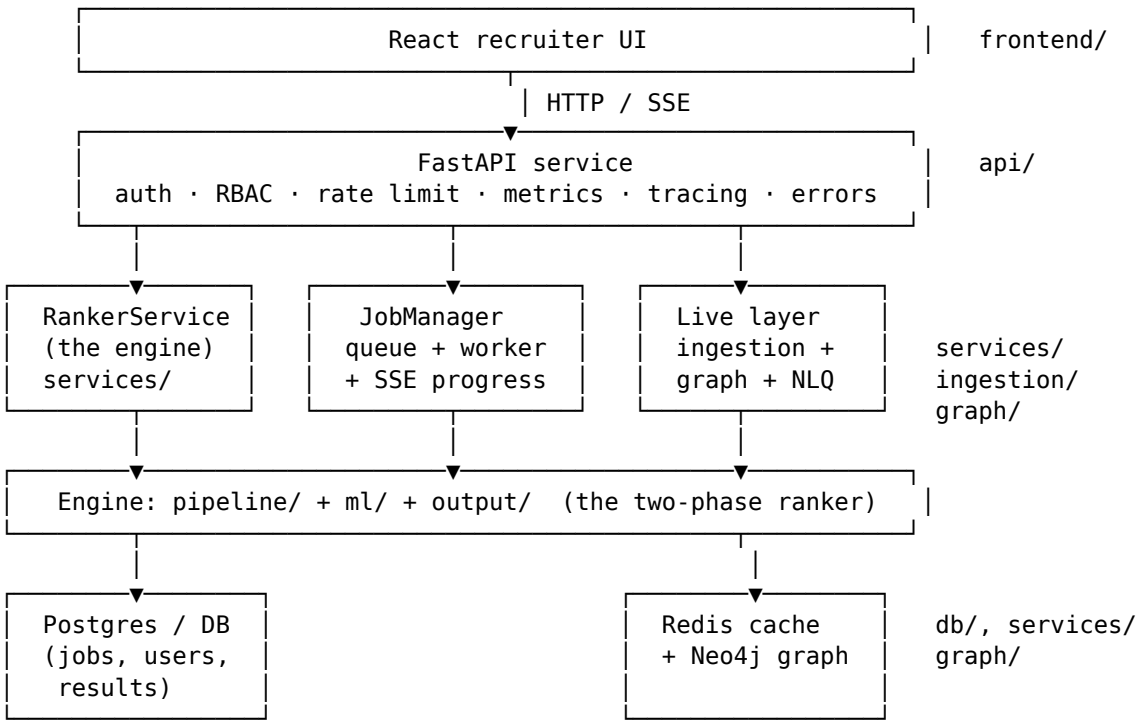
Phase 2 in detail (`rank.py`)

```
1. Load artifacts (feature_matrix.pkl, the model, embedding meta).
2. predict_fit() ──► the ML "fit" score per candidate (loads model.pkl).
3. merge_final_scores() ──► combine into one number:

     core   = 0.40·ml_fit + 0.35·retrieval + 0.25·behavioural
     base   = (1 − w_head)·core + w_rerank·rerank + w_tour·tournament   ← head only
     final  = base · location_factor · signal_modifier + evidence_bonus

4. Force honeypots and gated candidates to score 0 (they can never reach top 100).
5. Sort, take the top 100.
6. reasoning_generator ──► a one-line human justification per pick.
7. submission_writer ──► write submission.csv and run the official validator.
```

There are actually two ways into Phase 2:

- **Fast path** — the requested candidates are already in the precomputed matrix, so it just reads cached features. This is the graded path (milliseconds).
- **Live path** — brand-new candidates not seen at precompute time (e.g. an API caller submits fresh profiles). It computes their features on the fly. Slower, but flexible. The fast path falls back to this automatically.

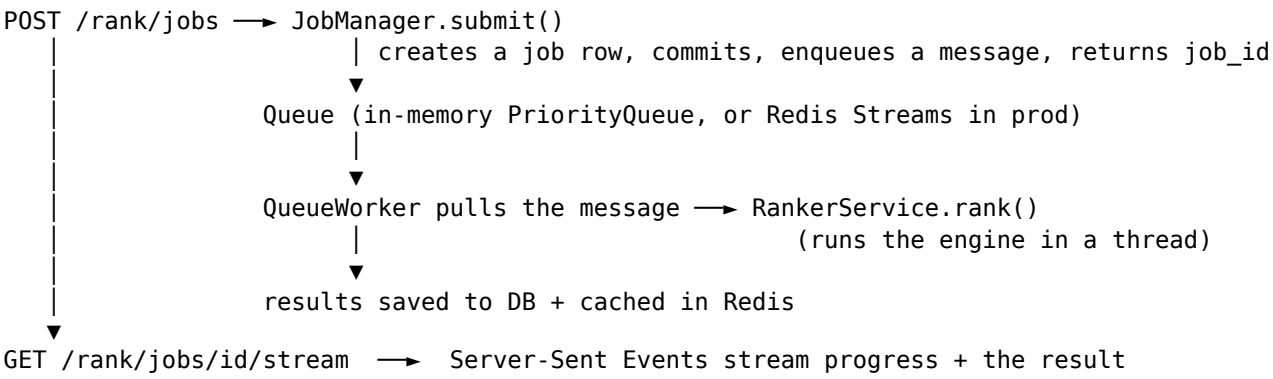
How the four layers stack



Crucially, the **engine doesn't depend on the service.** `precompute.py` and `rank.py` run from the command line with nothing else installed. The API, DB, cache, and live layer are wrappers *around* the engine, so a failure in any of them can't break the graded submission. This is "microservice-style isolation" applied within one codebase: each concern is a separate module with a fallback.

Data flow for a single ranking request (production)

```
Recruiter clicks "Run ranking"
   │
   ▼
```



The worker is **idempotent** (a duplicate message for a finished job is a no-op) and **at-least-once** (failures retry, then dead-letter). At scale, the worker moves to its own machine fleet without changing the API — that's the whole point of putting a queue in the middle.

The graceful-degradation ladder

Every external dependency is optional. The system picks the best available option and silently drops to the next:

Capability	Best	→	→	Floor
Embeddings	sentence-transformers (GPU)	(CPU)	—	NumPy hashing embedder
Keyword search	rank-bm25	—	—	hand-written inverted index
ML ranker	XGBoost LambdaMART	LightGBM	scikit-learn	linear formula
Reranker	CrossEncoder (GPU/ONNX)	(CPU)	—	lexical BM25-style scorer
JD parsing / labels	Gemini LLM	—	—	rule-based heuristics
Database	Postgres (asyncpg)	—	—	SQLite (aiosqlite)
Cache	Redis	—	—	in-memory LRU
Knowledge graph	Neo4j	—	—	in-memory graph store
Background jobs	Celery + Redis	—	—	run inline (eager)

This is why the project runs end-to-end in a bare sandbox with only NumPy and pandas, *and* scales up to a full GPU + Postgres + Redis + Neo4j deployment — same code, different rungs of the ladder.

Continue to 03_features.md for each feature and the tech behind it, or jump to 04_file_by_file.md for the module reference.